Python Scalar User-Defined Function (UDF) examples — a tour from unit conversion to cross-system keying

> This package contains **eight scalar UDFs** that together illustrate the simplest and oldest category of PyFlink user-defined function: a pure, row-at-a-time function callable from Flink SQL. They are the Python siblings of the eight Java classes in `../java/`, and — where it matters — produce byte-identical output for the same input. They are grouped into three themes — starting with the tiniest possible "Hello, UDF" and building up to the scalar UDFs you are most likely to reach for in a real streaming pipeline.
>
> **Unit-conversion primer** (the "Hello, UDF" pair)
>
> - **`celsius_to_fahrenheit`** — converts a `DOUBLE` Celsius temperature to its Fahrenheit equivalent using `F = (C × 9/5) + 32`.
> - **`fahrenheit_to_celsius`** — converts a `DOUBLE` Fahrenheit temperature to its Celsius equivalent using `C = (F − 32) × 5/9`.
>
> **PII & data-governance UDFs**
>
> - **`pseudonymize_pii`** — deterministic, keyed HMAC-SHA256 pseudonym of a PII value, domain-separated by a namespace argument. The keying material is loaded from AWS Secrets Manager (with a plain-env fallback) via the shared `secrets_resolver` helper.
> - **`resolve_identity`** — un-keyed SHA-256 of a normalized identity signal (email trimmed+lower-cased, phone digits-only, etc.). Use as a portable join key across systems that don't share a secret.
> - **`tokenize_pan`** — tokenizes a Primary Account Number (PAN) into a `{BIN}{HMAC-hex middle}{last-4}` shape, taking the Flink pipeline out of PCI DSS scope while preserving BIN and last-4 for fraud analytics and reconciliation. Same Secrets Manager / env-var resolution as `pseudonymize_pii`.
> - **`mask_pii_regex`** — detects PII entities (emails, IPs, US SSNs, credit cards with Luhn) in free-form text using Microsoft Presidio's regex recognizers and masks every match with `****`. The sibling Java UDF `MaskPiiRegex` produces byte-identical output for the same input — the parity test is what keeps both implementations honest.
>
> **Pipeline-reliability UDFs**
>
> - **`dedup_key`** — computes a stable idempotency key from a Python `list[str]` of business-level fields using length-prefix-encoded SHA-256. Use in a `ROW_NUMBER() OVER (PARTITION BY dedup_key …)` pattern to defang at-least-once duplicates.
> - **`consistent_bucket`** — maps a key to `[0, num_buckets)` using Kafka's 32-bit `murmur2` — the exact same hash Kafka's default partitioner uses, so Flink output co-partitions with Kafka producers, database shards, and cache slots.
>
> All eight UDFs ship in the same `src/` package, are registered as separate Flink functions via `CREATE FUNCTION ... LANGUAGE PYTHON`, and run in PyFlink's separate Python worker processes (one per task slot) under the Apache Beam SDK harness. Together they demonstrate how the `pyflink.table.udf.ScalarFunction` API lets you write plain, deterministic Python methods that Flink can call once per input row — no state, no timers, no operator graph surgery required.

Table of Contents

- 1.0 What is a Scalar UDF?
  - 1.1 Where scalar UDFs sit in the UDF hierarchy
  - 1.2 How PyFlink discovers and calls a scalar UDF
  - 1.3 Determinism and `NULL` handling
  - 1.4 One-time setup in `open(function_context)`
  - 1.5 How a Python UDF actually runs (Beam worker, not JVM)
- 2.0 Temperature conversions — the "Hello, UDF" pair
  - 2.1 `celsius_to_fahrenheit`
  - 2.2 `fahrenheit_to_celsius`
- 3.0 PII & data-governance UDFs
  - 3.1 `pseudonymize_pii` — irreversibly hide PII while keeping joins intact
  - 3.2 `resolve_identity` — portable canonical key across partner systems
  - 3.3 `tokenize_pan` — drop a Flink pipeline out of PCI DSS scope
  - 3.4 `mask_pii_regex` — Presidio-backed free-form text PII masking with Java/Python parity
- 4.0 Pipeline-reliability UDFs
  - 4.1 `dedup_key` — stable idempotency key for at-least-once streams
  - 4.2 `consistent_bucket` — Kafka-compatible murmur2 partitioning
- 5.0 Comparing the UDFs in this package
- 6.0 Project layout, build, and deployment
  - 6.1 Source layout (`src/`) and `uv` reproducibility
  - 6.2 Local run via `uv run` (no cluster)
  - 6.3 CP cluster deployment via the `cp-flink-python` image
  - 6.4 CC cluster deployment via a `confluent_flink_artifact` ZIP
  - 6.5 Secrets resolution: AWS Secrets Manager (LocalStack) with env-var fallback
- 7.0 Resources

1.0 What is a Scalar UDF?

A **Scalar UDF** is a user-defined function that takes zero or more input values from a single row and returns exactly one value. It is the closest Flink equivalent of a built-in SQL function like `UPPER()`, `ABS()`, or `CAST()`: the function sees one row at a time, has no memory of prior rows, and produces exactly one output value per invocation.

1.1 Where scalar UDFs sit in the UDF hierarchy

PyFlink mirrors Flink's four UDF categories. Scalar UDFs are the simplest, and most of the heavier categories build on the same `ScalarFunction` / `eval()` shape:

UDF category	Rows in	Rows out	State	Example use case
Scalar (this example)	1 value from 1 row	1 value	❌	Unit conversion, string cleaning, deterministic math, keyed hashing, partitioning
Table-valued	1 row	0..N rows	❌	Exploding a CSV column into multiple rows

UDF category	Rows in	Rows out	State	Example use case
Aggregate	N rows	1 value	✅	`GEOMEAN()`, weighted median
Process Table Function (PTF)	Table argument	0..N rows	✅ (set semantics)	Sessionization, stateful enrichment

Choose a scalar UDF when the computation depends only on the current row's values and can be expressed as a pure function.

1.2 How PyFlink discovers and calls a scalar UDF

A PyFlink scalar UDF is a Python class that extends `pyflink.table.udf.ScalarFunction` and defines a method named `eval`. Unlike Java (where Flink reflects on overloaded `eval` methods at SQL plan time to pick a signature), PyFlink relies on an explicit `udf(...)` wrapper that pins the input/output `DataTypes` so the SQL planner has no work to do:

- The method name **must** be `eval` (lower-case).
- The wrapped instance must be exposed at module level so SQL `CREATE FUNCTION` can reference it as `<module>.<symbol>`.
- Input/output types are **declared at wrap time**, not inferred from Python annotations.

The simplest module in this package follows the minimal skeleton:

```
from pyflink.table.udf import ScalarFunction, udf
from pyflink.table import DataTypes


class CelsiusToFahrenheit(ScalarFunction):
    def eval(self, celsius):
        if celsius is None:
            return None
        return (celsius * 9.0 / 5.0) + 32.0


celsius_to_fahrenheit = udf(
    CelsiusToFahrenheit(),
    input_types=[DataTypes.DOUBLE()],
    result_type=DataTypes.DOUBLE(),
)
```

At SQL level you register the wrapped symbol as a function:

```
CREATE FUNCTION celsius_to_fahrenheit
    AS 'scalar_udf.celsius_to_fahrenheit.celsius_to_fahrenheit'
    LANGUAGE PYTHON;
```

...and call it like any built-in scalar function:

```
SELECT sensor_id,
       celsius_temperature,
       celsius_to_fahrenheit(celsius_temperature) AS fahrenheit_temperature
  FROM celsius_reading;
```

The `'<package>.<module>.<symbol>'` form is the key Python-vs-Java difference. In Java you reference a class (`scalar_udf.CelsiusToFahrenheit`); in Python you reference the **wrapped `udf(...)` symbol inside a module inside the `scalar_udf` package** because that wrapper carries the type information PyFlink needs.

1.3 Determinism and `NULL` handling

Every UDF in this package is **deterministic**: the same input always produces the same output. Determinism is not just documentation — it is a contract that lets the Flink planner safely cache, reorder, and replay calls during failure recovery. For pseudonymization and bucketing, determinism is also what makes joins and `GROUP BY` work on the UDF's output. If you need a non-deterministic scalar UDF (e.g., one that reads the wall clock), set `deterministic=False` on the `udf(...)` wrapper.

Every `eval` method also preserves SQL `NULL` semantics by returning `None` when the input is `None`. This keeps the UDF transparent to the rest of the query: a `NULL` stays `NULL` through the pipeline rather than silently becoming a surprise value.

1.4 One-time setup in `open(function_context)`

Several UDFs in this package (`pseudonymize_pii`, `tokenize_pan`, `resolve_identity`, `dedup_key`) override `open(function_context)` to set up state once per Python worker before the first `eval()` call. This is where you:

- Load secrets via `secrets_resolver.resolve(...)` (AWS Secrets Manager via `*_SECRET_ID` env var, or plain `*_SECRET` env var fallback — used by `pseudonymize_pii` and `tokenize_pan`).
- Pre-construct cryptographic primitives (cached `bytes` keys for `hmac.new(...)`, reusable `hashlib.sha256()` factories) so each row doesn't pay the allocation cost.

Because PyFlink runs each parallel instance in a single-threaded Python worker, cached state on `self` does not need synchronization — it is effectively process-confined.

1.5 How a Python UDF actually runs (Beam worker, not JVM)

The runtime model is the biggest behavioral difference from the Java siblings and is worth internalizing before you ship a Python UDF to production:

- The Flink TaskManager (a JVM) does **not** execute Python code directly. For each task slot that hits a Python UDF, the TaskManager launches a separate **Apache Beam SDK harness** Python worker process and exchanges row data with it via gRPC.
- Each input row is **serialized JVM → Python**, the `eval` returns, and the result is **serialized Python → JVM**. That serialization round-trip is the dominant cost compared to a Java UDF that the planner inlines into the operator chain.
- Python UDFs run in PyFlink's `python.execution-mode = process` by default. PyFlink also supports `thread` mode (in-JVM via JEP), which removes the gRPC hop but constrains what Python libraries you can use — check the PyFlink docs before switching.

- Practical implication: prefer Java UDFs for hot-path arithmetic that runs billions of times per second; reach for Python when you specifically need a Python-only library (Presidio, NumPy, pandas, scikit-learn) — `mask_pii_regex` is the strongest example in this package because Presidio's recognizer set lives in Python.

2.0 Temperature conversions — the "Hello, UDF" pair

These two UDFs are the teaching case: the smallest possible shape of a scalar UDF that still does something real. They take a `DOUBLE`, return a `DOUBLE`, have no state, and handle `NULL` the SQL-standard way.

2.1 `celsius_to_fahrenheit`

Applies the standard formula `F = (C × 9/5) + 32`. The wrapped `udf(...)` declares `DataTypes.DOUBLE()` for both the input and the result, so the function plugs in wherever a `DOUBLE` column flows.

Source: `src/scalar_udf/celsius_to_fahrenheit.py`

```
Kafka (celsius_reading)
        │
        ▼
  ┌──────────────────────┐
  │ celsius_reading      │   Flink SQL source table (JSON / earliest-offset)
  └──────────────────────┘
            │
            ▼
  celsius_to_fahrenheit(celsius_temperature)    ← stateless scalar call per
row
            │
            ▼
  ┌────────────────────────┐
  │ celsius_to_fahrenheit  │   Flink SQL sink table → Kafka
(celsius_to_fahrenheit)
  └────────────────────────┘
```

The UDF is invoked once per input row; no keying, no state, no windowing. Because the body is a simple arithmetic expression, the per-row JVM↔Python serialization cost dominates — a useful baseline to compare against the heavier UDFs further down.

2.2 `fahrenheit_to_celsius`

Applies the inverse formula `C = (F − 32) × 5/9`. Same signature (`Double → Double`), same determinism and `NULL`-preservation guarantees as `celsius_to_fahrenheit` — only the arithmetic differs.

Source: `src/scalar_udf/fahrenheit_to_celsius.py`

```
Kafka (fahrenheit_reading)
        │
        ▼
  ┌──────────────────────┐
```

```
    │ fahrenheit_reading    │    Flink SQL source table (JSON / earliest-offset)
    └──────────────────────┘
              │
              ▼
   fahrenheit_to_celsius(fahrenheit_temperature)   ← stateless scalar call per
row
              │
              ▼
    ┌──────────────────────────┐
    │ fahrenheit_to_celsius    │    Flink SQL sink table → Kafka
(fahrenheit_to_celsius)
    └──────────────────────────┘
```

3.0 PII & data-governance UDFs

These four UDFs cover the overlapping but distinct problems of **hiding PII from analysts**, **resolving the same real-world entity across systems**, **reducing the PCI DSS blast radius of a Flink pipeline handling payment cards**, and **masking detected PII entities in free-form text**. The first three share a common implementation shape — they `open()` once to set up a cryptographic primitive, then hash per row; the fourth (`mask_pii_regex`) sits apart because it detects PII spans rather than transforming a known-type column, and it destroys the original value rather than producing a joinable surrogate. The design choices differ deliberately across all four:

- Secret or no secret?
- Per-call normalization or raw bytes?
- Hex blob or a structured PAN-shaped output?

The comparison table in §5 tabulates those differences.

3.1 `pseudonymize_pii` — irreversibly hide PII while keeping joins intact

Computes `HMAC-SHA256(secret, namespace || 0x1F || value)` and returns a 64-character hex string. The HMAC key is loaded in `open()` via `secrets_resolver.resolve("PII_PSEUDONYM")`: when `PII_PSEUDONYM_SECRET_ID` is set, the value is fetched from AWS Secrets Manager (honoring `AWS_ENDPOINT_URL_SECRETSMANAGER` so LocalStack works in dev); otherwise the resolver falls back to the `PII_PSEUDONYM_SECRET` env var. Either way the raw PII cannot be recovered by anyone who sees only the pseudonymized column — yet the same PII always pseudonymizes to the same value, so `JOIN`, `GROUP BY`, and `COUNT DISTINCT` continue to work on the pseudonymized column.

The mandatory `namespace` argument (`'email'`, `'phone'`, `'ssn'`) provides **domain separation**: the same underlying string pseudonymizes to a different value in each namespace, which prevents an analyst from silently linking an email column to a contact-email column in a different table.

Source: `src/scalar_udf/pseudonymize_pii.py`

```sql
CREATE FUNCTION pseudonymize_pii
    AS 'scalar_udf.pseudonymize_pii.pseudonymize_pii'
    LANGUAGE PYTHON;

SELECT pseudonymize_pii('email', email_address) AS email_pseudonym,
```

```
        pseudonymize_pii('phone', phone_number)  AS phone_pseudonym,
        order_total
  FROM orders;
```

> **Why HMAC instead of `SHA-256(salt ‖ value)`?** Prepending a fixed salt to a hash is vulnerable to length-extension attacks on Merkle-Damgård hashes like SHA-256. HMAC is specifically designed to resist that. Per-value **random** salts are deliberately *not* used because they would break the join-stability that makes the output useful for analytics.

3.2 `resolve_identity` — portable canonical key across partner systems

Computes `SHA-256(type || 0x1F || normalized_value)` after normalizing the value per identity type (email → trim+lower-case, phone → digits-only, default → trim+lower-case). The output is a 64-character hex string that two partner systems can derive independently without sharing a secret — that is the whole point of the **un-keyed** hash, and the deliberate inverse of the design choice made in `pseudonymize_pii`.

The tradeoff: un-keyed means the output is feasible to brute-force if the plaintext domain is small (e.g., all US phone numbers), so a resolved identity key is a **linkage identifier**, not a privacy pseudonym. When you need both portability *and* irreversibility, you need an out-of-band mechanism (e.g., a shared secret via DCR / clean-room infrastructure).

Source: `src/scalar_udf/resolve_identity.py`

```
CREATE FUNCTION resolve_identity
    AS 'scalar_udf.resolve_identity.resolve_identity'
    LANGUAGE PYTHON;

-- Join customers from the CRM to events from the web analytics system
-- using a canonical email-based identity key:
SELECT c.customer_id,
       e.event_name,
       e.event_ts
  FROM crm_customers    c
  JOIN analytics_events e
    ON resolve_identity('email', c.email_address)
     = resolve_identity('email', e.user_email);
```

> **Phone-number caveat:** the UDF's phone rule is "keep digits 0-9", which means `(415) 555-0134` (10 digits) and `+1-415-555-0134` (11 digits) resolve to **different** keys. Country-code inference requires a numbering-plan database like Google's `phonenumbers` Python port; **E.164-normalize phone values upstream** before passing them to this UDF.

3.3 `tokenize_pan` — drop a Flink pipeline out of PCI DSS scope

Tokenizes a Primary Account Number (PAN) into a token of the form `{BIN (first 6)}{HMAC-SHA256 hex middle}{last 4}` — same length as the input PAN, but the middle digits are replaced with keyed hex characters. A 16-digit PAN `4111111111111111` with an HMAC secret known only to the tokenization service becomes something like `411111a3c92b1111`.

This is **irreversible deterministic tokenization**: the token cannot be converted back to a PAN without the secret (no vault lookup is involved), so downstream systems that hold only tokens fall out of PCI DSS scope. PCI DSS §3.3 permits displaying first-6 and last-4; preserving them keeps fraud models, card-type analytics, and statement-reconciliation UIs working without any code change. The HMAC secret is sourced through the same `secrets_resolver` helper as `pseudonymize_pii` (uses `PAN_TOKENIZATION_SECRET_ID` / `PAN_TOKENIZATION_SECRET`).

Source: `src/scalar_udf/tokenize_pan.py`

```
CREATE FUNCTION tokenize_pan
    AS 'scalar_udf.tokenize_pan.tokenize_pan'
    LANGUAGE PYTHON;

SELECT order_id,
       tokenize_pan(card_number) AS card_token,
       SUBSTRING(tokenize_pan(card_number), 1, 6) AS bin,
       order_total
  FROM orders;
```

> **What this is *not*:** a vault-backed reversible tokenizer. If you need to charge the card later, use your PSP's network-token API — those require network I/O and state, which belong in an async function or an external service, not in a scalar UDF. And using this UDF is a necessary-but-not-sufficient PCI step; you still need secret custody, rotation, and network segmentation between the "has PAN" and "has only tokens" zones.

3.4 `mask_pii_regex` — Presidio-backed free-form text PII masking with Java/Python parity

Detects PII entities in free-form text using Microsoft Presidio's regex recognizers and replaces every matched span with `****`. Supported entity types are **email, IP address (v4/v6), US SSN, and credit card** — with Luhn checksum for cards and Presidio's invalidate-result logic for SSNs (rejects all-same-digit, mismatched delimiters, zero groups, known sample SSNs like `123456789` / `078051120`).

Unlike the other PII UDFs in this package, `mask_pii_regex` is **destructive** (you cannot recover the original value from a masked output) and **non-deterministic across JOINs** (every match becomes the same `****`, so the output is not joinable on the PII column). Use it when the pipeline input is genuinely free-form text — chat messages, support tickets, unstructured notes — where you don't know which columns contain which entity types. For structured per-column PII handling, `pseudonymize_pii` is the right tool.

This Python implementation is the **reference** for the cross-language parity contract: it instantiates Presidio's predefined recognizers directly (no `AnalyzerEngine`, no spaCy NER) so the runtime stack is just `presidio-analyzer` + `presidio-anonymizer`. The sibling `MaskPiiRegex.java` ports those exact patterns into Java and runs a shared fixture set through both implementations to confirm byte-identical output.

Source: `src/scalar_udf/mask_pii_regex.py` (Python, reference) and `MaskPiiRegex.java` (Java, port).

```sql
CREATE FUNCTION mask_pii_regex
    AS 'scalar_udf.mask_pii_regex.mask_pii_regex'
    LANGUAGE PYTHON;

SELECT ticket_id,
       mask_pii_regex(customer_message) AS redacted_message
  FROM support_tickets;
```

> **Java/Python parity is enforced by cross-checking** the Java implementation against this Python sibling. The test ports Presidio's patterns from `presidio-analyzer==2.2.359` verbatim into Java, applies an identical longest-match-wins greedy overlap resolution in both languages, and runs a shared fixture set through both. All 16 fixtures produce byte-identical output; if you bump the Presidio version in `pyproject.toml`, re-run the parity check because Presidio's pattern set occasionally changes between releases.

> **Why no `AnalyzerEngine` + spaCy?** Spinning up Presidio's full stack pulls in spaCy and a language model (~50 MB), neither of which has a JVM equivalent — so the parity contract would be impossible to keep. Direct recognizer use is the explicit price of cross-language parity.

> **If you need NER** (names, organizations, locations that can't be regex-matched), use Presidio's `AnalyzerEngine` with `SpacyNlpEngine` in a Python-only UDF. There is no equivalent NER stack for the JVM that produces matching output, so that path gives up Java parity by design — document it clearly in your data-lineage metadata so downstream consumers know which rows are Java-reproducible and which aren't.

4.0 Pipeline-reliability UDFs

These two UDFs address two recurring operational problems in streaming pipelines: **duplicates** (because Kafka is at-least-once and sources get replayed on restart) and **partition skew** (because a sink, shard, or cache has a fixed partition count that upstream and downstream must agree on).

4.1 `dedup_key` — stable idempotency key for at-least-once streams

Takes a Python `list[str]` (Flink SQL `ARRAY<STRING>`) of the business-level fields that identify a logical event (`source_system`, `event_type`, business id, occurrence time, etc.) and returns a 64-character SHA-256 hex digest of the **length-prefix-encoded** field sequence. The key is stable across retries, replays, and partition changes, so a downstream `ROW_NUMBER() OVER (PARTITION BY dedup_key …)` is all it takes to defang at-least-once duplicates.

The non-obvious design choice is **length-prefix encoding** rather than a naive separator. With a separator like `|`, the two inputs `["a|b", "c"]` and `["a", "b|c"]` would hash to the same key because both concatenate to `a|b|c`. Length-prefix (`[4-byte BE length][UTF-8 bytes]` per field) is unambiguous no matter what bytes appear inside the fields, so two different field lists can never collide.

`None` fields are encoded as the length marker `0xFFFFFFFF` with no following bytes, making them distinct from empty strings (`0x00000000` with zero following bytes). The UDF does *not* normalize values (no trim, no case-folding) — normalization would hide real bugs from source systems that emit the "same" event with different formatting. The byte-level encoding is identical to the Java `DedupKey` sibling so both implementations produce the same digest for the same input.

Source: `src/scalar_udf/dedup_key.py`

```
CREATE FUNCTION dedup_key
    AS 'scalar_udf.dedup_key.dedup_key'
    LANGUAGE PYTHON;

-- Dedupe by keeping the earliest occurrence of each idempotency key:
INSERT INTO orders_clean
SELECT *
  FROM (
    SELECT dedup_key(ARRAY[
             source_system,
             event_type,
             CAST(business_id AS STRING),
             CAST(occurred_at AS STRING)
           ]) AS idempotency_key,
           *,
           ROW_NUMBER() OVER (
             PARTITION BY dedup_key(ARRAY[
               source_system, event_type,
               CAST(business_id AS STRING),
               CAST(occurred_at AS STRING)
             ])
             ORDER BY event_ts
           ) AS rn
      FROM orders_raw
  )
 WHERE rn = 1;
```

4.2 `consistent_bucket` — Kafka-compatible murmur2 partitioning

Maps a string key to a non-negative bucket index in `[0, num_buckets)` using the **same** 32-bit `murmur2` hash that Kafka's default partitioner uses (`org.apache.kafka.common.utils.Utils#murmur2`). This Python implementation ports the algorithm bit-for-bit so a Flink-computed bucket always matches the Kafka-producer partition for the same key and partition count, regardless of which language wrote the producer.

"Consistent" here means **cross-system consistent**, not "consistent hashing" in the Karger / Dynamo sense. There is no universal best partition hash — the right hash is the one that matches the system you need to co-partition with. For Kafka producers and most JVM streaming tools, that is `murmur2`.

Source: `src/scalar_udf/consistent_bucket.py`

```
CREATE FUNCTION consistent_bucket
    AS 'scalar_udf.consistent_bucket.consistent_bucket'
    LANGUAGE PYTHON;

-- Database shard routing: same customer always lands on the same shard
INSERT INTO orders_shard_0
SELECT * FROM orders_stream
 WHERE consistent_bucket(CAST(customer_id AS STRING), 16) = 0;
```

```sql
-- Deterministic 10% A/B test cohort (same user_id always in the same arm)
SELECT *
  FROM clickstream
 WHERE consistent_bucket(user_id, 100) < 10;

-- Deterministic ~5% sampling (same event_id always sampled or always
dropped)
SELECT *
  FROM raw_events
 WHERE consistent_bucket(event_id, 100) < 5;
```

> **Hash compatibility is strict:** Kafka's `murmur2` is *not* interchangeable with `murmur3`, with Spark's `HashPartitioner` (which uses Java `Object.hashCode()`), or with Python's built-in `hash()` (which is randomized per process by default). Those hashes share a name or a family, but their constants differ and they produce different bucket assignments. If your target system isn't Kafka, verify which hash it uses before relying on this UDF for co-partitioning.

5.0 Comparing the UDFs in this package

All eight modules share the same scalar-UDF contract; the table below tabulates where their design choices diverge.

UDF	Input signature	Output	Keyed?	Reversible?	Primary purpose
`celsius_to_fahrenheit`	`DOUBLE`	`DOUBLE`	N/A	Yes (inverse UDF)	Unit conversion
`fahrenheit_to_celsius`	`DOUBLE`	`DOUBLE`	N/A	Yes (inverse UDF)	Unit conversion
`pseudonymize_pii`	`(STRING namespace, STRING value)`	`STRING` (64-char hex)	✅ HMAC-SHA256	❌	Irreversibly hide PII from analysts
`resolve_identity`	`(STRING type, STRING raw)`	`STRING` (64-char hex)	❌ SHA-256	❌ (but brute-forceable in small domains)	Portable join key across partner systems
`tokenize_pan`	`STRING pan`	`STRING` (PAN-shaped)	✅ HMAC-SHA256	❌	PCI DSS scope reduction
`mask_pii_regex`	`STRING text`	`STRING` (with `****` replacements)	❌ regex + Luhn / SSN validation	❌ (destructive mask)	Free-form text PII masking (Presidio-backed)

UDF	Input signature	Output	Keyed?	Reversible?	Primary purpose
`dedup_key`	`ARRAY<STRING> fields`	`STRING` (64-char hex)	❌ SHA-256	❌	Idempotency key for at-least-once streams
`consistent_bucket`	`(STRING key, INT numBuckets)`	`INT [0, N)`	❌ murmur2	❌	Kafka-compatible partitioning / bucketing

Cross-cutting contract every UDF honors:

Aspect	Value
Base class	`pyflink.table.udf.ScalarFunction`
`eval` method name / visibility	`eval`, instance method
Type registration	Explicit via `udf(instance, input_types=..., result_type=...)` wrapper
State	None
Determinism	Deterministic (default)
`None` input →	`None` output
Rows in → rows out	1 → 1
SQL registration	`CREATE FUNCTION <name> AS '<module>.<symbol>' LANGUAGE PYTHON`

All eight UDFs ship as modules inside the `src/scalar_udf/` Python package and are registered as separate Flink functions via `CREATE FUNCTION ... LANGUAGE PYTHON` statements that reference different `scalar_udf.<module>.<symbol>` paths through the same installed package.

6.0 Project layout, build, and deployment

This section covers the bits that have no direct analog in the Java sibling package: the source layout, the `uv`-managed reproducibility model, and the two deployment paths.

6.1 Source layout (`src/`) and `uv` reproducibility

```
python/
├── pyproject.toml              # PEP 621 project metadata, dependency decls,
setuptools build config
├── uv.lock                     # Pinned, hash-verified resolution of every
transitive dep
├── .python-version             # Pins CPython 3.11 for `uv` and the Docker
```

```
build
└── src/
    ├── run_job.py              # Local-only driver; lives outside the
package so
    │                           #   `python run_job.py` still puts src/ on
sys.path
    └── scalar_udf/             # The installable Python package (all UDFs +
secrets helper)
        ├── __init__.py         # Marks scalar_udf as a package
        ├── celsius_to_fahrenheit.py
        ├── fahrenheit_to_celsius.py
        ├── pseudonymize_pii.py
        ├── resolve_identity.py
        ├── tokenize_pan.py
        ├── mask_pii_regex.py
        ├── dedup_key.py
        ├── consistent_bucket.py
        └── secrets_resolver.py # Shared helper for Secrets Manager + env-var
fallback
```

`uv` manages the venv, locks transitive dependencies, and reproduces the same environment in CI, in the Dockerfile, and on a developer laptop. The `cp-flink-python` Docker image runs `uv sync --frozen --no-install-project --no-dev` against the committed `uv.lock` to build the deps-only venv layer, then a second `uv pip install --no-deps .` installs the `scalar_udf` package itself into the venv — so at SQL time `import scalar_udf.<module>` resolves the same way it does under `uv run` locally.

Notable pinned versions in `pyproject.toml`:

- `apache-flink==2.1.1` — matched to the Flink runtime version in the cluster.
- `presidio-analyzer==2.2.359` / `presidio-anonymizer==2.2.362` — pinned to the exact PyPI releases the Java `MaskPiiRegex` regex patterns were ported from. Bumping these requires re-running the cross-language parity test.
- `boto3>=1.36,<2` — used by `secrets_resolver` only when `*_SECRET_ID` env vars opt into AWS Secrets Manager. Unused on the env-var-only fallback path.

6.2 Local run via `uv run` (no cluster)

`src/run_job.py` is a small driver that registers `celsius_to_fahrenheit` and `fahrenheit_to_celsius` against in-memory `VALUES` tables and prints the results — enough to smoke-test a UDF without a cluster:

```
cd examples/scalar_udf/python
uv sync                                  # builds .venv from uv.lock
uv run --directory src python run_job.py
```

Use the same pattern to script ad-hoc tests of any other UDF in the package. Because PyFlink's mini-cluster runs the Python worker in-process, you can drop a `breakpoint()` into an `eval` method and step through it with `pdb`.

6.3 CP cluster deployment via the `cp-flink-python` image

For minikube/Confluent Platform deployment, the `cp-flink-python` image extends `confluentinc/cp-flink:2.1.1-cp1-java21` with Python 3.11, the prebuilt venv, and the `scalar_udf` package installed into `/opt/flink/python-udf/.venv/lib/python3.11/site-packages/` on every JM/TM pod. The image is built into the minikube docker daemon (no registry push needed):

```
make build-cp-flink-python-image
FLINK_IMAGE=cp-flink-python:2.1.1-cp1-java21 make flink-deploy
make deploy-cp-scalar-udf-python
```

The deploy script (`scripts/deploy-cp-scalar-udf-python.sh`) sets the Flink session-level `python.executable` and `python.client.executable` SQL options to point at the baked-in venv's interpreter, then runs the `CREATE FUNCTION ... LANGUAGE PYTHON` + `INSERT INTO` statements through the SQL Client. There is **no `python.files` SET** — the `scalar_udf` package lives in the venv's site-packages, so PyFlink's Python worker resolves `import scalar_udf.<module>` via the normal import system.

> **The Dockerfile installs the whole `scalar_udf` package** via `uv pip install --no-deps .`, so every UDF in `src/scalar_udf/` is deployable end-to-end without touching the image build. To register a new UDF on a running cluster, just add `DROP FUNCTION` / `CREATE FUNCTION` / `INSERT INTO` blocks for it to `scripts/deploy-cp-scalar-udf-python.sh` — no Dockerfile edits required.

6.4 CC cluster deployment via a `confluent_flink_artifact` ZIP

Confluent Cloud for Apache Flink accepts Python UDFs as a **source distribution wrapped in a ZIP**, uploaded via a `confluent_flink_artifact` resource with `content_format = "ZIP"` and `runtime_language = "Python"`. Because CC's Python worker runtime accepts `apache-flink==2.1.1` (the same version the CP session cluster runs), the CC artifact is built directly from **this same `pyproject.toml`** — no separate CC-only project is needed.

```
make build-scalar-udf-cc-python    # produces
examples/scalar_udf/python/dist/scalar_udf_python-<ver>.zip
# or, end-to-end:
make deploy-cc-scalar-udf CONFLUENT_API_KEY=<key> CONFLUENT_API_SECRET=
<secret>
```

Behind the `build-scalar-udf-cc-python` target:

1. `uv build --sdist` runs against this directory, producing `dist/scalar_udf_python-<ver>.tar.gz` via setuptools. The `[tool.setuptools.packages.find]` scope of `include = ["scalar_udf*"]` deliberately excludes `src/run_job.py` so the local-only driver doesn't end up in the uploaded artifact.
2. `zip -FS -j` wraps the tarball into `dist/scalar_udf_python-<ver>.zip`.
3. The cc_deploy terraform uploads the ZIP via `confluent_flink_artifact`, and `CREATE FUNCTION … AS 'scalar_udf.celsius_to_fahrenheit.celsius_to_fahrenheit' LANGUAGE PYTHON USING JAR 'confluent-artifact://…'` resolves the same `<package>.<module>.<symbol>` path as the CP deploy — **one source tree, one set of function addresses, two artifact formats**.

> ⚠️ **Confluent Cloud Python UDFs are an Early Access feature** at the time of writing. Your CC organization must have Python UDFs enabled for `terraform apply` on the `confluent_flink_artifact` with `runtime_language = "Python"` to succeed. See `../cc_deploy/README.md` §3.0 for the workaround when the feature isn't enabled.

6.5 Secrets resolution: AWS Secrets Manager (LocalStack) with env-var fallback

`pseudonymize_pii` and `tokenize_pan` resolve their HMAC keys through `src/scalar_udf/secrets_resolver.py`, which checks two sources in order:

1. **`<BASE_NAME>_SECRET_ID` set** → fetch the secret via boto3, honoring `AWS_ENDPOINT_URL_SECRETSMANAGER` so a LocalStack endpoint can be used in dev/minikube without changing UDF code.
2. **Otherwise** → fall back to the `<BASE_NAME>_SECRET` env var (keeps unit tests and outside-of-Kubernetes runs working).

The minikube workflow seeds LocalStack with random or supplied values via the `make localstack-up` target (see the project root `Makefile`), and the `flink-basic-deployment.yaml` pod template wires `AWS_ENDPOINT_URL_SECRETSMANAGER`, `AWS_REGION`, dummy creds, and the per-UDF `*_SECRET_ID` env vars onto the Flink containers — so a cluster-deployed `pseudonymize_pii` / `tokenize_pan` will pick up the LocalStack-seeded secret without any code change.

`boto3` is imported lazily inside the resolver, so jobs that only use the env-var fallback path do not pay its import cost (and do not need it installed at all on the worker).

7.0 Resources

- Apache Flink User-defined Functions
- PyFlink `ScalarFunction`
- PyFlink Python UDFs guide
- PyFlink Python execution mode (`process` vs `thread`)
- Apache Beam Python SDK harness (the runtime that executes Python UDFs out-of-process)
- Create a User-Defined Function with Confluent Cloud for Apache Flink
- `uv` — fast Python package & project manager (the venv + lockfile tool used here)
- RFC 2104: HMAC — Keyed-Hashing for Message Authentication (algorithm behind `pseudonymize_pii` and `tokenize_pan`)
- PCI DSS v4.0 §3.3 — PAN Display (rationale for the first-6 + last-4 format in `tokenize_pan`)
- ITU-T E.164 — The international public telecommunication numbering plan (phone normalization contract for `resolve_identity`)
- Apache Kafka `Utils.murmur2` (reference implementation `consistent_bucket` ports verbatim)
- Microsoft Presidio — predefined regex recognizers (pattern source for `mask_pii_regex`)
- LocalStack — AWS cloud emulator (in-cluster Secrets Manager backend used by the minikube workflow)
- AWS SDK for Python (`boto3`) — Secrets Manager (client used by `secrets_resolver` when `*_SECRET_ID` is set)